

DEVWORKS

HARNESS

GUIA COMPLETO DE USO

Da instalação ao ciclo contínuo de desenvolvimento com Plan, Task Builder, Ralph, gates, dashboard e failover de modelos.

PUBLICO

Desenvolvedores e equipes que desejam usar o harness em projetos novos ou legados, com diferentes assinaturas e engines de IA.

Edicao	Data	Escopo
MVP operacional	28/09/2026	Uso local e compartilhamento interno

A regra central é simples: o agente prepara ou executa; a equipe revisa, valida e decide o commit.

O FLUXO EM UMA PAGINA

1. Visão geral

O DEVWORKS Harness organiza o trabalho em quatro passos. Para um projeto novo, o ponto de partida é um PRD. Para um legado, o ponto de partida é a documentação do código real.

Etapa	Responsabilidade	Saída
Plan	Transforma PRD ou solicitação em fases maiores, semelhantes a releases.	SPEC.md, PLAN.md e visão de fases
Task Builder	Detalha uma fase em tarefas pequenas, ordenadas e testáveis.	PHASES.md executável
Ralph	Executa uma fase por vez, com sessões novas, correções e quatro gates.	Código alterado, logs e estado
Humano	Valida critérios, revisa o diff e cria o commit.	Checkpoint seguro para o próximo ciclo

CICLO RECOMENDADO

Plan -> Task Builder -> Ralph -> gates verdes -> revisão manual -> testes manuais -> commit -> próxima fase.

O commit manual é o ponto de segurança entre execuções. O Ralph não faz stage, commit, reset ou alteração de exclusões do Git.

Estado real deste MVP

Componente	Situação
Ralph e quatro engines	Implementado e testado
Gates, histórico, lock, TUI e dashboard web	Implementado
Política Antigravity com fallback de modelo	Implementada no Ralph
Skills portáteis e Task Builder	Implementados: init, plan, task-builder e ai-context
Plugin Claude	Comandos legados preservados como adaptador

ANTES DE INSTALAR

2. Pré-requisitos e segurança

O harness trabalha sobre um repositório Git local. Ele pode executar comandos e editar arquivos sem intervenção durante uma fase. Use apenas em projetos confiáveis e prefira uma branch própria.

	Verificacao
☐	Git instalado e repositório inicializado.
☐	Bash 4 ou superior para execução funcional completa dos testes do harness.
☐	Python 3 para o dashboard web local.
☐	Pelo menos uma engine instalada e autenticada.
☐	Comando real de testes do projeto conhecido.
☐	Branch de trabalho criada antes da primeira execução.

SEGREDOS

Nunca coloque tokens, chaves ou senhas no repositório, no PHASES.md ou nos prompts. Use o login oficial da CLI, variáveis de ambiente seguras ou o cofre corporativo.

Permissões de execução

A sessão implementadora recebe permissão para editar e executar comandos. O Gate 3 usa uma sessão restrita e somente leitura. Essa separação reduz o risco de o próprio implementador aprovar o que fez.

Checklist mínimo antes de rodar

	Verificacao
☐	Confirmar diretório, branch e arquivo PHASES.md.
☐	Revisar mudanças já existentes na árvore.
☐	Definir o comando de teste com --test-cmd quando a detecção automática não for suficiente.
☐	Revisar migrations, integrações externas e operações irreversíveis antes de autorizar a fase.

REPOSITORIO CENTRAL

3. Instalação do harness

Mantenha uma cópia central do DEVWORKS Harness fora dos projetos atendidos. Os scripts são chamados a partir da raiz do projeto-alvo.

```
git clone <url-interno-ou-github>/devworks-harness.git
cd devworks-harness
git switch main
chmod +x scripts/*.sh scripts/engines/*.sh
```

Valide o harness antes de compartilhar

```
./scripts/check-shell.sh
./scripts/test-ralph.sh
git diff --check
```

Uso a partir de outro projeto

```
cd /caminho/do/projeto-alvo
git switch -c feat/minha-entrega
/caminho/devworks-harness/scripts/ralph.sh --help
```

IMPORTANTE

O Ralph é independente e está pronto para as quatro engines. Os quatro Skills canônicos já estão empacotados; rode check-skill-drift antes de distribuir uma nova versão. O plugin Claude continua como adaptador compatível.

Estrutura relevante

Caminho	Finalidade
scripts/ralph.sh	Orquestrador principal
scripts/engines/	Adaptadores Codex, Claude, OpenCode e Antigravity
config/model-policy.tsv	Política de modelos do Antigravity
scripts/ralph-watch.sh	Painel de terminal
scripts/ralph-web.sh	Dashboard web local
skills/	Fonte canônica: init, plan, task-builder e ai-context
commands/ e agents/	Adaptador de compatibilidade do plugin Claude

ESCOLHA POR ACESSO E CONTEXTO

4. Engines suportadas

Engine é a CLI que executa o trabalho. Modelo é a inteligência selecionada dentro daquela engine. Trocar a engine não exige reescrever o PHASES.md.

Engine	Executável	Melhor uso inicial	Autenticação
Codex	codex	Equipe com assinatura ChatGPT/Codex; execução e revisão de código.	Login com conta ChatGPT ou credencial corporativa
Claude Code	claude	Planejamento, arquitetura e uso completo do plugin atual.	Claude Pro/Max, Console, Bedrock ou Vertex
OpenCode	opencode	Equipe que precisa escolher entre vários provedores.	Login por provedor; conferir política corporativa
Antigravity	agy	Assinatura Google com acesso a Gemini e, quando disponível, Claude.	Google OAuth ou projeto Google Cloud

Instalação rápida por engine

```
# Codex
npm install -g @openai/codex
codex

# Claude Code
npm install -g @anthropic-ai/claude-code
claude

# OpenCode
npm install -g @opencode/cli
opencode auth login

# Antigravity - macOS/Linux
curl -fsSL https://antigravity.google/cli/install.sh | bash
agy
```

NÃO CONFUNDA

Claude é a família de modelos e Claude Code é uma engine. Antigravity é outra engine e pode expor Claude Opus/Sonnet e Gemini conforme a assinatura. A lista real deve ser confirmada com agy models.

UM TESTE POR MAQUINA

5. Configuração e teste das engines

Engine	Verificar	Executar no Ralph
Codex	codex --version e login interativo	--engine codex
Claude	claude --version e claude	--engine claude
OpenCode	opencode --version e opencode auth list	--engine opencode
Antigravity	agy --version e agy models	--engine antigravity

```
./scripts/ralph.sh --engine codex --test-cmd 'npm test' PHASES.md
./scripts/ralph.sh --engine claude --test-cmd 'composer test' PHASES.md
./scripts/ralph.sh --engine opencode --test-cmd 'pytest' PHASES.md
./scripts/ralph.sh --engine antigravity --profile standard PHASES.md
```

Como escolher

	Verificacao
[]	Use Codex quando a equipe já possui acesso e quer uma engine direta para implementação.
[]	Use Claude Code quando o fluxo também depende dos comandos e agentes do plugin DEVWORKS.
[]	Use OpenCode quando a flexibilidade de provedor for o fator principal.
[]	Use Antigravity quando a assinatura Google é a fonte principal e o fallback entre modelos deve ser automático.

PORTABILIDADE

O contrato do Ralph é o PHASES.md. A mesma fase pode ser executada por outra engine, desde que a primeira execução tenha sido encerrada e o diff atual tenha sido revisado.

QUALIDADE ONDE IMPORTA; VELOCIDADE NO VOLUME

6. Política de modelos do Antigravity

A política inicial reserva Claude Opus e Gemini 3.1 Pro High para decisões pesadas. Gemini 3.8 Flash atende a execução recorrente. Os nomes são identificadores reais da CLI e podem mudar; confirme periodicamente com `agy models`.

Papel/perfil	Primário	Fallback 1	Fallback 2
Plan	Claude Opus 4.6 Thinking	Gemini 3.1 Pro High	Gemini 3.8 Flash High
Task Builder	Claude Opus 4.6 Thinking	Gemini 3.1 Pro High	Gemini 3.8 Flash High
Execução routine	Gemini 3.8 Flash Medium	Gemini 3.8 Flash High	Gemini 3.1 Pro High
Execução standard	Gemini 3.8 Flash High	Gemini 3.1 Pro High	Claude Opus 4.6 Thinking
Execução complex	Gemini 3.1 Pro High	Claude Opus 4.6 Thinking	Gemini 3.8 Flash High
Verificação	Claude Sonnet 4.6	Gemini 3.8 Flash High	-

```
# Rotina simples
./scripts/ralph.sh --engine antigravity --profile routine PHASES.md

# Padrão recomendado
./scripts/ralph.sh --engine antigravity --profile standard PHASES.md

# Arquitetura/refatoração difícil
./scripts/ralph.sh --engine antigravity --profile complex PHASES.md
```

Quando ocorre limite de uso do modelo atual, o Ralph tenta o próximo modelo da mesma rota e continua a mesma fase. Se todas as rotas estiverem esgotadas, ele aguarda o reset. Falha de teste ou gate vermelho não troca o modelo automaticamente: ela abre um ciclo normal de correção.

No MVP atual, o Ralph aplica automaticamente somente as rotas `execution-*` e `verify`. Para Plan e Task Builder, selecione a rota definida na política dentro da interface da engine; a Skill preserva o mesmo contrato de saída.

CONFIGURAÇÃO CENTRAL

Edite `config/model-policy.tsv` para ajustar a ordem corporativa. Evite espalhar nomes de modelos em scripts e documentação de projeto.

CADA PASSO DEIXA UMA SAIDA REVISAVEL

7. Artefatos e contrato entre etapas

Artefato	Responsável	Uso seguinte
PRD ou solicitação	Produto / solicitante	Entrada do Plan
SPEC.md	Plan	Requisitos e critérios de aceite
PLAN.md	Plan	Arquitetura, riscos, dependências e fases
PHASES.md	Task Builder	Input direto do Ralph
AGENTS.md e docs/agents/*	ai-context	Contexto real de projeto legado
.phases/runs/*	Ralph	Logs, prompts, gates e histórico
Commit	Desenvolvedor	Checkpoint entre ciclos

Formato mínimo do PHASES.md

```
## Phase 1: Nome da fase

- [ ] Task 1: resultado verificável
- Arquivos prováveis: caminho/arquivo.ext
- Critério de aceite: condição binária
- Teste: comando ou cenário

- [ ] Task 2: próximo resultado
```

REGRA DE FORMATO

Cada fase de nível superior deve usar exatamente ## Phase N: Título. Subfases podem usar ### Phase N.M:. Uma task deve ser pequena o bastante para ter critério binário e evidência verificável.

PRD -> RELEASES -> TASKS -> CÓDIGO

8. Projeto novo

Em um projeto novo, o Plan define as fases maiores. Elas funcionam como releases ou marcos. O Task Builder só detalha a fase que será executada agora, evitando planejar em excesso.

	Verificacao
[]	Criar o repositório e uma branch inicial.
[]	Escrever ou anexar o PRD com objetivo, público, regras, restrições e critérios de aceite.
[]	Executar o Plan com Claude Opus; usar Gemini 3.1 Pro High como fallback.
[]	Revisar arquitetura, dependências, riscos e ordem das fases.
[]	Escolher a próxima fase e detalhá-la no Task Builder.
[]	Executar com Ralph; perfil standard por padrão, complex quando necessário.
[]	Validar, revisar o diff e commitar.
[]	Repetir o ciclo para a próxima fase ou melhoria.

Prompt de Plan

Use o modo Plan. Leia o PRD em docs/PRD.md.
Defina arquitetura, dependências, riscos, critérios de aceite e fases maiores.
As fases devem funcionar como releases. Não implemente código.

Prompt de Task Builder

Leia SPEC.md e PLAN.md. Detalhe somente a próxima fase em PHASES.md.
Crie tasks pequenas, ordenadas, com arquivos prováveis, critérios binários,
testes, dependências e riscos. Não implemente código.

REVISÃO OBRIGATÓRIA

O Task Builder já é um Skill portátil. Revise o PHASES.md antes de chamar o Ralph: o arquivo é o contrato que define o escopo executável.

DOCUMENTAR O QUE EXISTE ANTES DE MUDAR

9. Projeto legado

No legado, não comece pelo PRD nem pela implementação. Primeiro gere contexto a partir do código real. O ai-context não deve usar a pasta .spec como fonte; ele documenta apenas o que está implementado.

	Verificacao
[]	Abrir o repositório correto e registrar branch, stack e comandos existentes.
[]	Executar ai-context para gerar AGENTS.md, CLAUDE.md e docs/agents/*.
[]	Revisar arquitetura, domínio, contratos, modelo de dados e dependências documentadas.
[]	Descrever a manutenção desejada, incluindo comportamento atual e esperado.
[]	Executar o Plan usando a documentação real como referência.
[]	Detalhar somente a fase necessária no Task Builder.
[]	Executar Ralph, validar regressões e commitar manualmente.

```
# Usando qualquer engine após instalar os Skills
Use o Skill ai-context no repositório atual.
Use o Skill plan para: <descrição da manutenção>.
Use task-builder para a fase <N>.

# Depois de revisar o PHASES.md gerado
/caminho/devworks-harness/scripts/ralph.sh --engine antigravity \
--profile standard --test-cmd '<comando-real>' \
.spec/features/<slug>/phases/<NN>--<nome>/PHASES.md
```

PRESERVE O LEGADO

A documentação gerada é AS IS. O Plan descreve o TO BE. Não use ai-context para inventar arquitetura ideal nem para reescrever regras antigas sem evidência.

RESPONSABILIDADES DIFERENTES

10. Plan e Task Builder

Decisão	Plan	Task Builder
Escopo	Produto/feature e fases maiores	Uma fase por vez
Arquitetura	Decide abordagem e dependências	Respeita a abordagem aprovada
Granularidade	Release/marco	Task pequena e verificável
Riscos	Mapeia riscos amplos	Traduz em travas e sequência
Testes	Define estratégia	Define teste/evidência por task
Código	Não escreve	Não escreve

Critério de pronto do Plan

	Verificacao
[]	Fases têm objetivo, dependências, riscos e critérios de aceite.
[]	Decisões críticas e limites de escopo estão explícitos.
[]	A fase seguinte pode ser escolhida sem rediscutir toda a arquitetura.

Critério de pronto do Task Builder

	Verificacao
[]	Cada task produz um resultado observável.
[]	Arquivos prováveis e comandos de teste estão indicados.
[]	A ordem respeita dependências.
[]	A fase cabe em uma execução revisável.

EVITE

Não transforme o Task Builder em um segundo Plan. Se uma task exige uma nova decisão de arquitetura, volte ao Plan e atualize a fase antes de executar.

UMA SESSÃO NOVA POR FASE E CORREÇÃO

11. Execução com Ralph

O Ralph lê o PHASES.md, separa as fases e abre uma sessão nova para cada uma. Uma falha abre outra sessão com a causa real. A sessão anterior não é reutilizada.

```
./scripts/ralph.sh [opções] [arquivo]

--engine codex|claude|opencode|antigravity
--profile routine|standard|complex
--from N
--keep-going
--max-cycles N
--test-cmd '<comando>'
--no-verify
--dashboard
```

Exemplos

```
# Projeto Laravel
./scripts/ralph.sh --engine antigravity --profile standard \
--test-cmd 'vendor/bin/sail test' .spec/features/cadastro/PHASES.md

# Retomar da fase 3
./scripts/ralph.sh --engine codex --from 3 PHASES.md

# Refatoração complexa
./scripts/ralph.sh --engine antigravity --profile complex \
--max-cycles 2 PHASES.md
```

PADRÃO

Use standard para a maioria das fases. Use routine para trabalho repetitivo e bem delimitado. Use complex para arquitetura difícil, refatoração ampla ou diagnóstico com muitas dependências.

O QUE REALMENTE APROVA UMA FASE

12. Gates, correção e failover

Gate	Pergunta	Decisão
G0	A engine terminou de verdade?	Contrato específico da CLI e evento terminal
G1	A sessão alterou a árvore?	Sinal de escrita; não aprova sozinho
G2	A suíte de testes passou?	Ralph executa fora da sessão do agente
G3	Cada task existe no código?	Verificador read-only decide DONE/INCOMPLETE

Comportamentos diferentes

Evento	Resposta
Gate vermelho	Nova sessão de correção; consome um ciclo
Limite do modelo Antigravity	Próximo modelo da rota; mesma fase; não consome ciclo
Todas as rotas limitadas	Aguarda reset e repete a mesma fase
Fase falha após máximo de ciclos	Para por padrão; --keep-going continua
Mudança parcial	Permanece na árvore para diagnóstico e revisão

IDEMPOTÊNCIA

A troca de modelo reapresenta a fase completa. Por isso, tasks devem ser seguras para recheagem e efeitos externos precisam estar claramente sinalizados para revisão humana.

ACOMPANHAR SEM CONTROLAR

13. Dashboard e observabilidade

```
# Painel de terminal em outro terminal
./scripts/ralph-watch.sh /caminho/do/projeto

# Dashboard web local
./scripts/ralph-web.sh /caminho/do/projeto
# abrir http://127.0.0.1:7331
```

O dashboard é somente leitura e escuta apenas no computador local. Ele mostra run, engine, modelo, perfil, fase, task, ciclo, gates, atividade, duração, logs e histórico quando esses dados estão disponíveis no estado.

Arquivos de diagnóstico

Caminho	Conteúdo
.phases/current	Ponteiro para o run atual
.phases/runs/<run>/logs/	Saída de implementação e verificação
.phases/runs/<run>/prompts/	Prompt autocontido de cada sessão
.phases/runs/<run>/state/	Estado consolidado e progresso ao vivo
.phases/progress/	Fases concluídas por hash do input
.phases/.ralph.lock/	Trava contra dois Ralphs no mesmo repositório

LIMITAÇÃO INTENCIONAL

O dashboard não pausa, cancela, retoma nem altera a execução. Controle operacional permanece no terminal e no processo do Ralph.

CHECKPOINT HUMANO

14. Ciclo de validação e commit

Quando os gates ficarem verdes, ainda existe uma etapa humana. O objetivo é criar um ponto estável antes de continuar o desenvolvimento.

<pre>git status --short git diff --check git diff # executar validação manual do fluxo git add <arquivos-revisados> git commit -m 'feat: entrega a fase validada'</pre>	
	Verificacao
[]	Revisar todos os arquivos alterados, inclusive não rastreados.
[]	Confirmar que o diff corresponde somente à fase executada.
[]	Executar validação manual dos critérios de aceite relevantes.
[]	Conferir migrations, permissões, logs e rollback quando aplicável.
[]	Criar o commit somente depois da revisão.
[]	Iniciar o próximo ciclo a partir desse checkpoint.
POR QUE O COMMIT É MANUAL O commit registra uma decisão humana. Ele separa as execuções, facilita comparação, rollback e retomada, e impede que o agente consolide mudanças que ninguém revisou.	

QUANDO ALGO NÃO FUNCIONA

15. Diagnóstico rápido

Sintoma	Ação
CLI não encontrada	Instalar a engine e confirmar --version
Falha de autenticação	Abrir a CLI interativa e concluir o login oficial
Antigravity sem modelo	Executar agy models e atualizar model-policy.tsv
G0 vermelho	Ler o final do log da sessão e o evento terminal
G2 vermelho	Executar o mesmo comando de teste fora do Ralph
G3 vermelho	Ler o log verify; identificar tasks INCOMPLETE
Painel parado	Conferir run.tsv, live.tsv e marcadores RALPH-TASK
Run concorrente	Confirmar o processo ativo; não apagar lock de um processo vivo
Plano mudou	Encerrar o run, revisar PHASES.md e iniciar novo ciclo
Árvore já estava suja	Separar visualmente mudanças antigas e novas antes do commit

Validação do próprio harness

```
./scripts/check-shell.sh  
./scripts/test-ralph.sh  
git diff --check
```

NÃO MASCARAR

Não desligue testes, Gate 3 ou validações apenas para deixar a fase verde. Corrija a causa ou registre conscientemente a limitação.

COMEÇAR PEQUENO E PADRONIZAR

16. Adoção na empresa

	Verificacao
[]	Publicar o harness em um repositório interno com versão e changelog.
[]	Definir quais engines e assinaturas a empresa permite.
[]	Definir política padrão de modelos por papel e revisar nomes periodicamente.
[]	Criar um projeto piloto pequeno, com testes confiáveis e baixo risco.
[]	Treinar a equipe no ciclo Plan -> Task Builder -> Ralph -> revisão -> commit.
[]	Exigir branch própria e comando de testes em toda execução.
[]	Coletar falhas reais antes de aumentar a automação.
[]	Rodar check-skill-drift e testar instalação em uma máquina piloto antes de atualizar a distribuição.

Responsabilidades

Papel	Responsabilidade
Produto/solicitante	PRD, prioridade e critérios de aceite
Desenvolvedor	Plan, revisão de tarefas, execução e commit
Líder técnico	Política de engines/modelos, arquitetura e exceções
Segurança/infra	Segredos, permissões, rede e ambientes
Harness	Orquestração, gates, estado, logs e continuidade

PRIMEIRA META

Não tente automatizar toda a empresa de uma vez. Primeiro prove um ciclo completo e repetível em um projeto real, mantendo o commit manual como checkpoint.

COPIAR, ADAPTAR E EXECUTAR

17. Referência rápida

Projeto novo

```
PRD -> Plan (Opus / Pro High) -> revisar fases
-> Task Builder da próxima fase -> revisar PHASES.md
-> Ralph (Flash / perfil standard) -> gates
-> validação manual -> commit
```

Projeto legado

```
ai-context -> revisar documentação AS IS
-> Plan da manutenção -> Task Builder
-> Ralph -> gates -> regressão manual -> commit
```

Comandos Ralph

```
./scripts/ralph.sh --engine codex PHASES.md
./scripts/ralph.sh --engine claude PHASES.md
./scripts/ralph.sh --engine opencode PHASES.md
./scripts/ralph.sh --engine antigravity --profile standard PHASES.md
```

Decisão de modelo

Situação	Escolha inicial
Planejamento e decisões pesadas	Claude Opus; fallback Gemini 3.1 Pro High
Execução comum	Gemini 3.8 Flash High
Execução repetitiva	Gemini 3.8 Flash Medium
Refatoração/arquitetura complexa	Gemini 3.1 Pro High; fallback Opus
Verificação	Claude Sonnet; fallback Flash High

DOCUMENTAÇÃO CONSULTADA

18. Fontes e notas de versão

As instruções de instalação e autenticação abaixo foram conferidas em documentação oficial. A lista de modelos Antigravity foi também conferida na CLI instalada durante a preparação deste guia.

OpenAI - Codex CLI e login com ChatGPT

Anthropic - instalação do Claude Code

OpenCode - instalação e CLI

Google - Antigravity CLI, modelos e modo headless

INFORMAÇÃO VOLÁTIL

Modelos, planos, limites, comandos de autenticação e disponibilidade por assinatura podem mudar. Antes de um rollout corporativo, revise a documentação oficial e execute os comandos de versão/modelos em uma máquina piloto.

Limitações conhecidas desta edição

	Verificacao
[]	As Skills são portáteis por contrato; a interface de chamada ainda varia por engine.
[]	Plan e Task Builder usam a política de modelos como recomendação; a seleção de rota nessas interfaces ainda é manual.
[]	O failover de modelos está implementado para a engine Antigravity; outras engines mantêm seu próprio modelo configurado.
[]	O dashboard é local e somente leitura.

DEVWORKS Harness - guia de uso interno - edição MVP operacional